

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1.HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header.

The HTTP response header contains the following issues:

- Content-Length: -150 is **invalid** because Content-Length cannot be negative.
- The Content-Type and Content-Length headers should be written on separate lines.
- Other fields (HTTP/1.1 200 OK, Date, and Connection) are valid.

2. Analyze the issue with Content-Length and explain its impact.

- Content-Length specifies the size of the response body in bytes.
- A negative value (-150) is invalid according to HTTP standards.
- The browser cannot determine the correct size of the content.
- This may cause:
 - Failure to load the page correctly.
 - Truncated or incomplete content.
 - Connection or parsing errors.

3. Determine whether the status code matches the response correctness.

- Status code **200 OK** indicates the request was successful.
- Since the response header contains an invalid Content-Length, the response is **not completely correct**.
- Therefore, although the server reports success, the response is malformed.

4. Suggest corrections for the identified errors.

Correct HTTP Response:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

Improvements:

- Use a positive Content-Length value.
- Place each header on a separate line.
- Ensure the Content-Length matches the actual content size.

2. DOM Structure Analysis

1. Identify missing or improperly closed elements.

The HTML contains:

- Missing `</p>` closing tag.
- Missing `</div>` before the paragraph is closed properly.
- The `<p>` element is improperly nested.

2. Analyze how the browser will interpret this structure.

Modern browsers automatically correct invalid HTML by:

- Automatically closing the `<p>` tag.
- Creating a valid DOM tree internally.
- Displaying the page even though the HTML is incorrect.

Different browsers may produce slightly different DOM structures.

3. Determine the impact on rendering and layout.

- Page usually displays correctly.
- CSS styles may behave differently.
- JavaScript DOM manipulation may not work as expected.
- Browser compatibility issues may occur.

4. Suggest corrections to make the DOM valid.

```
<html>
<html>
<head>
  <title>TestPage</title>
</head>
<body>
<div>
  <p>Hello World</p>
</div>
</body>
</html>
```

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET	POST
Sends data in the URL	Sends data in the request body
Data is visible	Data is hidden from the URL
Limited data size	Can send large amount of data
Used for retrieving data	Used for submitting data

2. Analyze which method is more suitable for login and why.

POST is more suitable because:

- Credentials are not shown in the URL.
- Supports secure transmission with HTTPS.
- Passwords are not stored in browser history.
- Better suited for authentication.

3. Identify security risks in using GET for authentication.

Using GET may:

- Expose usernames and passwords in the URL.
- Save credentials in browser history.
- Store credentials in server logs.
- Allow sensitive data to be shared accidentally.

4. Suggest best practices for secure login implementation.

- Use **POST** instead of GET.
- Use **HTTPS**.
- Hash passwords on the server.
- Validate inputs.
- Use secure sessions.
- Implement CSRF protection.
- Use strong password policies.

5. HTML Code Inspection

1. Identify missing closing tags.

Missing tags are:

- `</h1>`
- `</p>`

2. Analyze issues with the `<img>` tag.

Problems:

- Missing alt attribute.
- Image may not display if the file is unavailable.
- Accessibility is reduced without alt.

3. Determine how browsers will handle these errors.

Browsers automatically:

- Close `<h1>` before starting `<p>`.
- Display the image if found.
- Ignore minor HTML errors.

However, different browsers may render the page slightly differently.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

5. Browser-Server Communication Flow

1. Interpret each step in the communication flow.

1. **User enters a URL** – The browser receives the website address.
2. **DNS lookup** – Converts the domain name into an IP address.
3. **HTTP request** – Browser sends a request to the web server.
4. **Server processing** – Server processes the request and retrieves the required resource.
5. **HTTP response** – Server returns HTML, CSS, JavaScript, images, etc.
6. **Browser rendering** – Browser displays the webpage to the user.

2. Identify where delays can occur.

Delays may occur during:

- DNS lookup.
- Internet/network latency.
- Server processing.
- Database queries.
- Downloading large files.
- Browser rendering.

3. Analyze the role of DNS in the process.

DNS (Domain Name System):

- Converts a domain name (e.g., www.example.com) into an IP address.
- Helps the browser locate the correct web server.
- Without DNS, users would need to remember IP addresses.

4. Suggest optimizations to improve performance.

- Enable DNS caching.
- Use browser caching.
- Compress files using Gzip/Brotli.
- Minify HTML, CSS, and JavaScript.
- Use a Content Delivery Network (CDN).
- Optimize images.
- Reduce HTTP requests.
- Improve server performance and database efficiency.

